Struttura applicazione

Description	Struttura applicazione in varie fasi

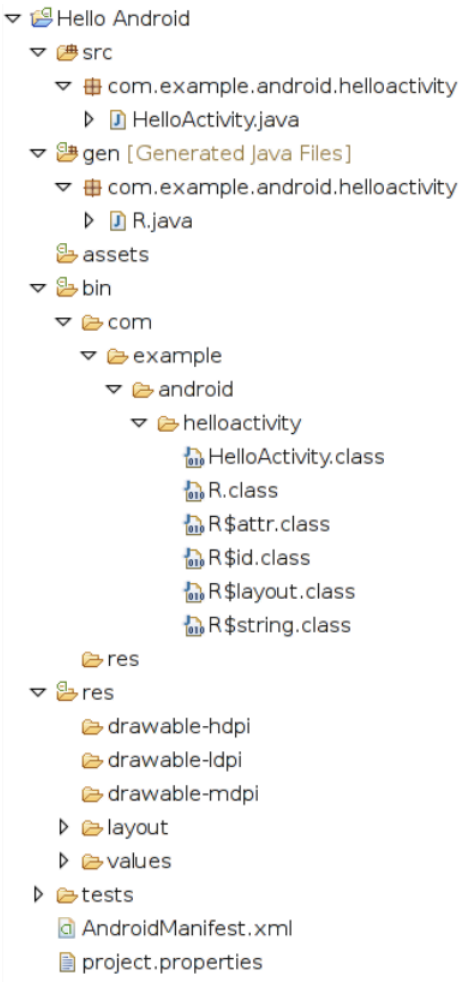
Struttura di un progetto

Un'applicazione in sviluppo è chiamata progetto. Il wizard di creazione di un nuovo
progetto Android dell'ADT crea lo scheletro per noi.

Solo alcune directory sono di interesse per lo sviluppatore, altre vengono generate. automaticamente

Su **ECLIPSE**

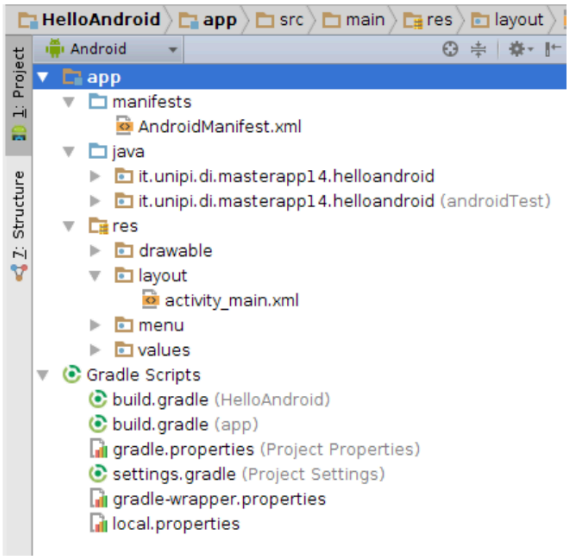
- ***src*** – sorgenti (.java, a volte .aidl o altri linguaggi)
- ***gen*** – sorgenti generati automaticamente (R.java)
- ***assets*** – file arbitrari, aggiunti al .apk
- ***bin*** – risultato della compilazione (risorse, .dex, .apk, ...)
- ***res*** – risorse note al runtime Android
- ***animator*** – animazioni basate su proprietà
- ***anim*** – animazioni basate su intercalazione
- *color* – colori
- ***drawable*** – immagini raster o vettoriali



- ***layout*** – descrizioni del layout della UI
- ***menu*** – menu usati dall'applicazione
- ***raw*** – file arbitrari (alternativa ad assets)
- ***values*** – costanti (stringa, interi, array, ecc.)
- ***xml*** – file XML arbitrari (incluse configurazioni)
- ***libs*** – librerie native custom
- ***AndroidManifest.xml*** – metadati sull'applicazione
- ***Altri*** .properties, .cfg, .xml – configurazioni varie

Su Android Studio

- ***manifests***
  - *AndroidManifest.xml* – metadati sull'applicazione
- ***java*** – sorgenti (.java, .kt, e unit test associati)
- ***assets*** – file arbitrari, aggiunti al .apk
- ***bin*** – risultato della compilazione (risorse, .dex, .apk, ...)
- ***res*** – risorse note al runtime Android



    - ***animator*** – animazioni basate su proprietà
    - ***anim*** – animazioni basate su intercalazione
    - ***color*** – colori
    - ***drawable*** – immagini raster o vettoriali
    - ***layout*** – descrizioni del layout della UI
    - ***menu*** – menu usati dall'applicazione
    - ***raw*** – file arbitrari (alternativa ad assets)
    - ***values*** – costanti (stringa, interi, array, ecc.)
    - ***xml*** – file XML arbitrari (incluse configurazioni)
- ***libs*** – librerie native custom
- ***Gradle Scripts*** – configurazioni di build
- ***Altri*** .properties, .cfg, .xml – configurazioni varie

File .apk

Un file con estensione `.apk` è un pacchetto di applicazioni Android. Contiene tutti gli elementi necessari per installare e eseguire un'applicazione su un dispositivo Android.

Manifesto dell'applicazione (`AndroidManifest.xml`)

Questo file XML contiene informazioni cruciali sull'app, come il nome del pacchetto, i componenti dell'app (attività, servizi, ricevitori di trasmissione), le autorizzazioni richieste, i filtri di intenti e altro ancora.

Il manifest è essenziale per Android perché fornisce informazioni sulle capacità e le configurazioni dell'app.

`resources.arsc` (Android Resources)

Questo file contiene le risorse precompilate dell'applicazione Android. È una parte essenziale per l'efficienza durante l'esecuzione dell'app, in quanto consente al sistema di accedere rapidamente alle risorse dell'app senza doverle elaborare o analizzare ulteriormente.

Contiene una tabella delle risorse che mappa gli identificatori (ID) delle risorse con i valori associati, consentendo un accesso rapido durante l'esecuzione dell'applicazione.

`classes.dex` (Dalvik Executable)

Questo file contiene il codice sorgente dell'app tradotto in bytecode dal compilatore Java o Kotlin. Dalvik è la macchina virtuale originariamente sviluppata da Google per eseguire applicazioni Android. In versioni più recenti di Android, è stata sostituita con l'Art (Android Runtime), ma il formato .dex è comunque utilizzato.

Questo file è essenziale per l'esecuzione dell'applicazione su dispositivi Android.

`AndroidManifest.xml`

Questo file XML contiene informazioni fondamentali sull'applicazione, come il nome del pacchetto, le attività, i servizi, i ricevitori di trasmissione, le autorizzazioni richieste e altro ancora.

È una parte cruciale del pacchetto APK in quanto fornisce al sistema Android informazioni sulle capacità e le configurazioni dell'app.

META-INF (Meta-Information)
Questa directory contiene informazioni aggiuntive sul pacchetto APK:

- `CERT.RSA`**:** Contiene il certificato del mittente, che può essere utilizzato per verificare l'autenticità dell'applicazione. Questo è particolarmente importante per garantire che l'app sia stata firmata da un'entità attendibile.
- `CERT.SF` **(Signature File):** Contiene la firma del certificato e le informazioni sulla struttura del pacchetto.
- `MANIFEST.MF` **(Manifest File):** Contiene informazioni sulle risorse del pacchetto, come nome del pacchetto, versione e intestazioni di firma.

Librerie Native (`lib/`)
Se l'applicazione utilizza librerie native scritte in linguaggi come C o C++, queste librerie saranno presenti in questa directory.

Assets (`assets/`)
Questa directory contiene file che possono essere recuperati utilizzando l'API AssetManager dell'API Android. Questi file non vengono compilati ma sono inclusi nel pacchetto APK.

File di configurazione (`META-INF/`)
Questa directory contiene file di configurazione, come CERT.RSA (certificato del mittente), CERT.SF (file firma del certificato) e MANIFEST.MF (informazioni sulla struttura del pacchetto).

Risorse Android (`res/`)
Questa directory contiene risorse come layout XML, immagini, icone, stringhe, stili, ecc.
Le risorse sono organizzate in diverse cartelle per diversi aspetti dell'applicazione.

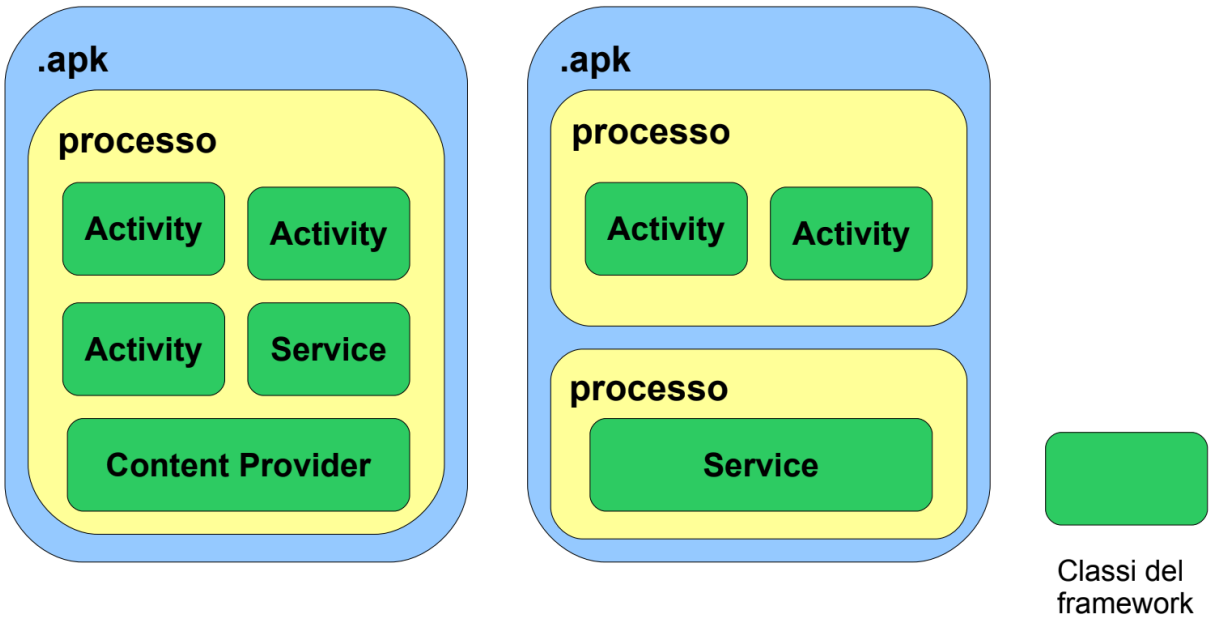
File di licenza e documentazione: Alcune applicazioni possono includere file di licenza open source o documentazione.

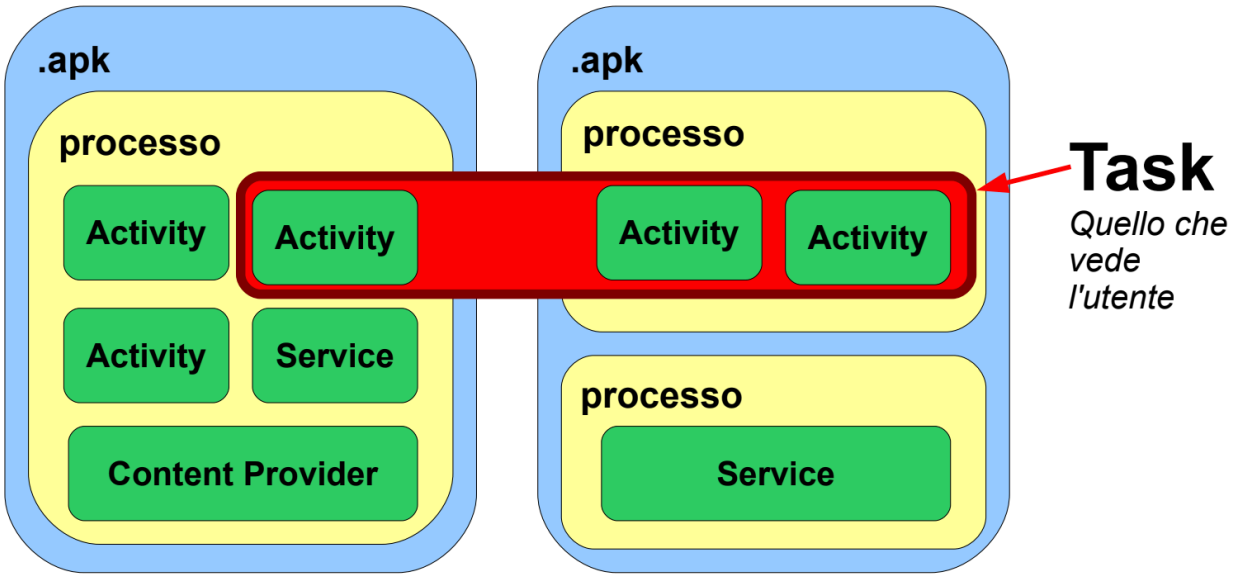
File di icone e immagini: L'applicazione può includere icone e immagini che vengono utilizzate per rappresentare l'applicazione nell'interfaccia utente e in altri contesti.

Altri file: A seconda delle esigenze dell'applicazione, potrebbero esserci altri file e risorse nel pacchetto APK.

Struttura in memoria

Una volta caricata in memoria, una applicazione è distinta in componenti, di norma abbiamo:
1 app = 1 processo = 1 VM = 1 thread





Ogni componente ha uno spazio dedicato in memoria, di norma è organizzata come segue:

1. **Zona di Codice (Code Section):**
   - **classes.dex:** Questo è il file contenente il bytecode delle classi Java o Kotlin convertito in formato ottimizzato per l'esecuzione su Dalvik (o ART). Viene caricato in memoria quando l'applicazione viene avviata.
2. **Heap:**
   - **Dalvik/ART Heap:** Questo spazio contiene gli oggetti allocati dinamicamente durante l'esecuzione dell'applicazione. Comprende l'heap per gli oggetti Java/Kotlin, gli oggetti nativi e la heap per gli oggetti "large" (oggetti che richiedono più spazio di memoria).
3. **Stack:**
   - **Stack di Chiamate (Call Stack):** Ogni thread dell'applicazione ha il proprio stack di chiamate. Questo spazio viene utilizzato per la gestione delle chiamate alle funzioni, con ogni chiamata di funzione che aggiunge un frame allo stack. Contiene i dati locali, i parametri della funzione e l'indirizzo di ritorno.
4. **Zona di Dati (Data Section):**

- **Variabili Globali e Statiche:** Questo spazio contiene le variabili globali e statiche dell'applicazione, inclusi i dati statici dichiarati all'interno delle classi.

5. **Zona dei Riferimenti (References Section):**
    - **Puntatori e Riferimenti:** Questa zona contiene i riferimenti a oggetti e risorse utilizzati dall'applicazione.
6. **Zona delle Risorse (Resources Section):**
    - **Risorse Precompilate (resources.arsc):** Le risorse precompilate dell'applicazione sono caricate in memoria per fornire un accesso più veloce durante l'esecuzione.
7. **Zona di Caricamento (Loading Section):**
    - **Librerie Native:** Se l'applicazione utilizza librerie native scritte in C o C++, queste librerie vengono caricate in memoria quando necessario.
8. **Zona dei File di Configurazione (Configuration Files Section):**
    - **AndroidManifest.xml:** Il manifesto dell'applicazione è letto in memoria per ottenere le informazioni di base e le configurazioni dell'app.